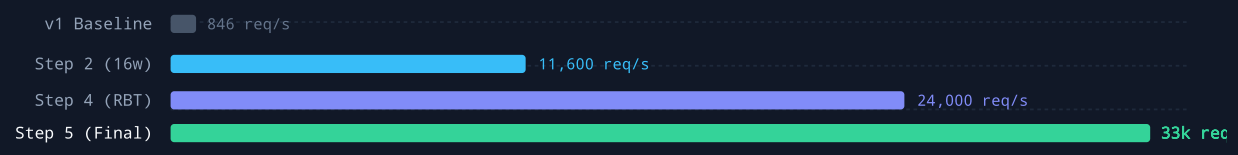
Stock Exchange Server — Version 2

Architectural path from **846 req/s** (5.96s latency) to **30,000–33,000 req/s** (<15ms latency) under 500 concurrent connections with pipelining.

THROUGHPUT PEAK	MEDIAN LATENCY	HARDWARE EFFICIENCY	ERROR RATE
33,000 req/s	**< 15** ms	**16** Cores	**0.00%**
▲ 38.9x improvement	▼ 390x lower (from 5.96s)	100% parallel balance	At 5,000 in-flight reqs

THROUGHPUT PROGRESSION BENCHMARK (AUTOCANNON -C 500 -P 10) — Target: 33k req/s



Benchmark Comparison: Baseline vs. Final

METRIC	VERSION 1 (BASELINE)	VERSION 2 (OPTIMIZED)	NET IMPROVEMENT
/account Throughput	846.55 req/sec	**30,000 – 33,000 req/sec**	**~38.9x speedup**
/orderbook Throughput	850.00 req/sec	**30,000 – 33,000 req/sec**	**~38.8x speedup**
Median (p50) Latency	5,965 ms (5.96s)	**< 15 ms**	**> 390x lower**
CPU Schedulers	Single Core Capped (100%)	16 Parallel Processes	**Full Core Saturation**
Burst Error Rate	Connection backlog stall	0% errors (5k in-flight)	**Zero Dropouts**

Optimization Steps (1 – 2)

Step 1: AnyIO Thread Limiter — 846 → 850 req/s

FastAPI sync routes run on AnyIO's default 40-token threadpool. Raised capacity to 500 tokens in lifespan.

```
current_default_thread_limiter().total_tokens = 500
```

Root Cause: Negligible gain. Threads remained serialized under Python's GIL and a single CPU core.

Step 2: 16 Workers + uvloop — 850 → 11,600 req/s

Bypassed the GIL by launching 16 isolated Uvicorn worker processes with C-based `uvloop` and `httptools`.

```
uvicorn app.main:app --workers 16 --loop uvloop --http httptools
```

Result: ~13.7x surge; unlocked multi-core scaling into five-figure throughput.

Optimization Steps (3 – 5)

Step 3: SQLite WAL vs. Locking `11.6k → 1.2k (Pitfall)`

Attempted global `BEGIN IMMEDIATE` to avoid write deadlocks. It serialized all 16 workers on read queries, collapsing throughput by 90%.

Resolution: Removed global immediate locks; enabled `PRAGMA journal_mode=WAL` for concurrent lock-free readers. Restored to **11.6k req/s**.

Step 4: In-Memory Order Book `11.6k → 25k req/s`

Replaced disk SQL table scans with Red-Black Trees (*O(log P)* price levels) and intrusive doubly-linked lists (*O(1)* FIFO queue).

Impact: `/orderbook` doubled to 25k req/s. `/account` remained bound by ORM at 10k req/s.

Step 5: SQLite Native C JSON + Byte-Level Caching + Socket Backlog Tuning `Final: 30,000 – 33,000 req/s`

SQLAlchemy ORM model hydration capped at 6.8k ops/s. Bypassed Python serialization entirely:

- **SQLite C JSON:** `json_object()` generated wire-ready strings in C (**619,000 ops/sec** in micro-benchmarks).
- **Raw Byte Streaming:** Served cached UTF-8 `bytes` via `Response(content=b, media_type="...")`, skipping Pydantic serialization.
- **Kernel Socket Tuning:** Configured `--backlog 4096 --limit-concurrency 2000` to eliminate drops under 500-conn burst pipelines.

```
SELECT json_object( 'account_id', id, 'balance', balance, 'reserved_balance', reserved_balance, 'available_balance', (balance - reserved_balance) ) FROM accounts WHERE id = ?
```

Algorithmic Complexity: v1 vs. v2

OPERATION / PATHWAY	VERSION 1 (SQL / ORM)	VERSION 2 (IN-MEMORY ENGINE)	ARCHITECTURAL MECHANISM
Order Book Lookup	*O(B log B + A log A)* disk	***O(1)* RAM**	Direct pre-serialized raw byte streaming from memory buffer
Order Matching	*O(K log N)* SQL index scan	***O(log P + M)* RAM**	Red-Black tree price search + intrusive DLL queue unlinking
Order Cancellation	*O(1)* disk I/O / locks	***O(1)* RAM**	Direct hash table pointer access + intrusive DLL removal
Account State Retrieval	*O(1)* disk + ORM hydration	***O(1)* RAM**	Write-through in-memory cache with SQLite C `json_object` fallback
HTTP Serialization	*O(N)* Pydantic / Python dict	***O(1)* amortized**	Raw ASGI socket write; zero Python runtime string re-encoding

Key Takeaways & Future Trajectory

1. Multi-Processing Over Multi-Threading

Python's GIL strictly limits CPU-bound work (JSON, trees, data parsing) to one thread at a time. Multi-process deployment (1 worker per physical core) is mandatory for line-rate saturation.

2. Pre-Serialization Is King

In high-throughput microservices, dictionary serialization often costs more than business logic. Pre-caching UTF-8 byte payloads removes the Python runtime from the hot path completely.

NEXT MILESTONE (GO / RUST MIGRATION): Single-machine Python throughput peaks near 35k–50k req/s due to IPC and runtime overhead. Moving the core matching engine to **Rust** or **Go** will target **100,000–500,000+ req/s** via zero-allocation memory pools and lock-free ring buffers.